RANCANGAN BASIS DATA RELASIONAL

Inventora

Sintaks SQL MySQL

CREATE seluruh tabel dan INSERT lima data untuk setiap tabel.

TABEL	DATA CONTOH	FOREIGN KEY	TARGET
14	70 baris	19 relasi	MySQL 8.0.16+

Sistem inventori dan pengadaan internal dengan pegawai, akun pengguna, supplier, persetujuan, penerimaan barang, pembayaran simulasi, payroll, dan anggaran departemen.

IDENTITAS	DIISI OLEH KELOMPOK
Nama kelompok	.....
Kelas / program studi	.....
Anggota dan NIM	....

Isi dokumen. Rancangan dan lingkup (hlm. 2), relasi (3-4), persiapan SQL (5), CREATE dan INSERT lengkap (6-19), query verifikasi (20-22), serta hasil pengujian (23).

Diuji pada MySQL Community Server 8.0.41. Seluruh 20 pemeriksaan lulus.

Lingkup dan keputusan desain

Rancangan mengikuti domain Inventora dan disiapkan sebagai artefak TUBES.

MODUL	TABEL
Organisasi dan akses	departemen, pegawai, pengguna
Master inventori	supplier, kategori_produk, produk
Pengadaan	purchase_order, detail_purchase_order, persetujuan_po
Penerimaan dan pembayaran	penerimaan_barang, detail_penerimaan, pembayaran_po
SDM dan perencanaan	payroll, anggaran_departemen

Integritas di tingkat database

Primary key mengidentifikasi setiap baris. Foreign key menjaga referensi induk dan detail, termasuk FK gabungan untuk memastikan produk penerimaan terdapat pada PO yang sama. UNIQUE mencegah username, payroll pegawai-periode, serta anggaran departemen-periode yang ganda. CHECK membatasi tanggal dan nominal di dalam satu baris.

Nilai uang memakai **DECIMAL(15,2)**. Subtotal detail, total PO, dan gaji bersih dihitung melalui generated columns. Harga transaksi serta komponen payroll merupakan snapshot historis sehingga perubahan master tidak mengubah riwayat.

Asumsi dan batas lingkup

Inventori satu lokasi. Stok pada produk adalah snapshot akhir. INSERT penerimaan tidak otomatis memperbarui stok; aplikasi memerlukan mutasi dan transaksi stok tersendiri.

Satu alur penuh per PO. Maksimal satu keputusan persetujuan, satu penerimaan, dan satu pembayaran penuh. Cicilan, penerimaan parsial berulang, dan riwayat percobaan pembayaran belum dimodelkan.

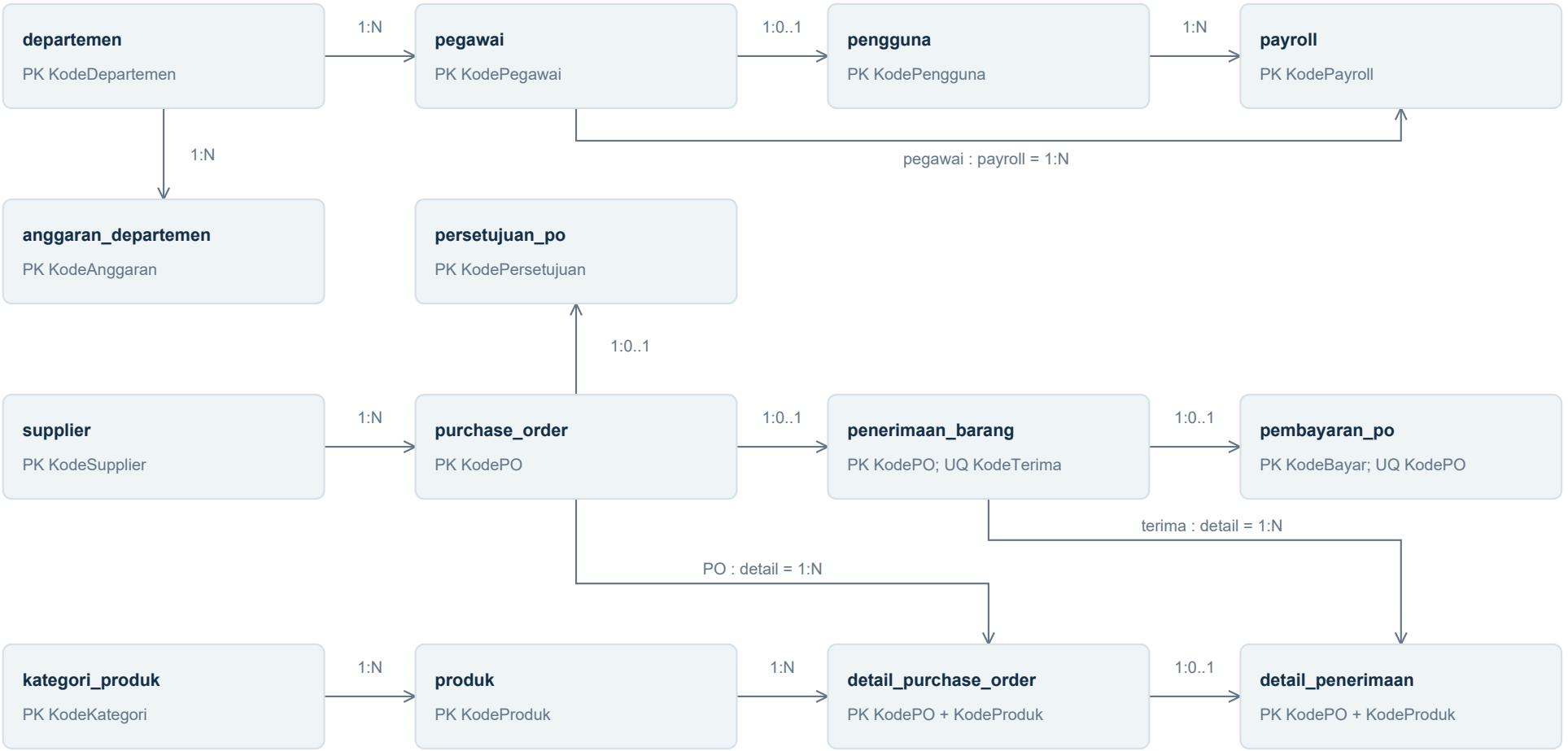
Aturan antartabel. Hak akses peran, transisi status, kesamaan total header-detail, jumlah bayar/terima, dan urutan tanggal antartabel memerlukan validasi aplikasi dalam transaksi atau prosedur/trigger tambahan. Data contoh telah diperiksa dengan query.

Identitas dan keuangan. AuthUID memetakan identitas eksternal, bukan password. Kontak dan pembayaran bersifat fiktif. Pajak, BPJS, serta PPh21 adalah nominal ilustrasi.

Aplikasi yang ada masih menggunakan Firebase. Paket ini menyediakan rancangan relasional untuk tugas dan tidak mengalihkan koneksi aplikasi. Tema inventori/pengadaan perlu sesuai proposal yang disetujui dosen; jumlah tabel tidak menjamin nilai tertentu.

Peta relasi Inventora

Panah dari induk ke anak. Label menyatakan jumlah anak per satu induk. FK pelaku transaksi dan departemen PO dirinci pada halaman berikutnya.



PK = primary key; UQ = unique key; 0..1 = opsional, paling banyak satu. Diagram ini merangkum relasi utama dan bukan representasi seluruh kolom.

Referensi antartabel

Seluruh FK memakai ON UPDATE RESTRICT dan ON DELETE RESTRICT.

NO.	KOLOM ANAK / FOREIGN KEY	KOLOM INDUK / REFERENSI
1	pegawai.KodeDepartemen	departemen.KodeDepartemen
2	pengguna.KodePegawai [UQ]	pegawai.KodePegawai
3	produk.KodeKategori	kategori_produk.KodeKategori
4	purchase_order.KodeSupplier	supplier.KodeSupplier
5	purchase_order.KodeDepartemen	departemen.KodeDepartemen
6	purchase_order.DibuatOleh	pengguna.KodePengguna
7	detail_purchase_order.KodePO	purchase_order.KodePO
8	detail_purchase_order.KodeProduk	produk.KodeProduk
9	persetujuan_po.KodePO [UQ]	purchase_order.KodePO
10	persetujuan_po.DisetujuiOleh	pengguna.KodePengguna
11	penerimaan_barang.KodePO [PK]	purchase_order.KodePO
12	penerimaan_barang.DiterimaOleh	pengguna.KodePengguna
13	detail_penerimaan.KodePO	penerimaan_barang.KodePO
14	detail_penerimaan.(KodePO, KodeProduk)	detail_purchase_order.(KodePO, KodeProduk)
15	pembayaran_po.KodePO [UQ]	penerimaan_barang.KodePO
16	pembayaran_po.DibayarOleh	pengguna.KodePengguna
17	payroll.KodePegawai	pegawai.KodePegawai
18	payroll.DicatatOleh	pengguna.KodePengguna
19	anggaran_departemen.KodeDepartemen	departemen.KodeDepartemen

Satu FK gabungan pada detail_penerimaan mencakup dua kolom. Karena itu jumlah constraint FK adalah 19, meskipun jumlah pasangan kolom referensi adalah 20. Tanda [UQ] atau [PK] pada kolom FK membatasi setiap induk menjadi paling banyak satu baris anak.

Membuat schema MySQL

Gunakan server MySQL 8.0.16+ dan schema inventora_tubes yang belum terisi.

```
CREATE DATABASE IF NOT EXISTS inventora_tubes
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE inventora_tubes;
SET SESSION sql_mode =
  'STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';
```

Urutan eksekusi

1. Jalankan persiapan di atas, lalu seluruh blok **CREATE TABLE** pada halaman 6-19 sesuai urutan.

2. Setelah semua tabel terbentuk, jalankan **START TRANSACTION;**.

3. Jalankan seluruh blok **INSERT** pada halaman 6-19 sesuai urutan, lalu **COMMIT;**.

4. Jalankan query verifikasi pada halaman 20-22. Setiap tabel harus berisi tepat lima baris.

```
START TRANSACTION;
-- Jalankan seluruh INSERT sesuai urutan dependensi.
COMMIT;
```

Pada dokumen, CREATE dan INSERT dikelompokkan per tabel agar mudah dibaca. File **inventora_tubes.sql** sudah disusun untuk eksekusi sekaligus: seluruh CREATE lebih dahulu, kemudian satu transaksi INSERT.

Melalui MySQL Workbench

Buka koneksi server, pilih **File > Open SQL Script**, buka inventora_tubes.sql, lalu jalankan seluruh skrip dengan tombol petir untuk semua pernyataan. Setelah selesai, buka dan jalankan verifikasi.sql.

File tidak berisi DROP. Jangan jalankan berulang pada schema yang telah terisi dan jangan gunakan --force. Bila terjadi error, hentikan dan periksa error pertama. DDL MySQL melakukan commit implisit; transaksi INSERT tidak membatalkan CREATE TABLE.

departemen

Unit organisasi

PK KodeDepartemen; NamaDepartemen unik. Induk pegawai, PO, dan anggaran.

CREATE TABLE

```
CREATE TABLE departemen (
  KodeDepartemen CHAR(5) PRIMARY KEY,
  NamaDepartemen VARCHAR(80) NOT NULL UNIQUE,
  Lokasi VARCHAR(80) NOT NULL
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO departemen (KodeDepartemen, NamaDepartemen, Lokasi) VALUES
  ('DPT01', 'Engineering dan IT', 'Lantai 1'),
  ('DPT02', 'Operasional', 'Lantai 2'),
  ('DPT03', 'Keuangan', 'Lantai 3'),
  ('DPT04', 'Sumber Daya Manusia', 'Lantai 4'),
  ('DPT05', 'Pergudangan', 'Gedung Gudang');
```

pegawai

Data induk pegawai

PK KodePegawai; FK departemen. GajiPokok di master dapat berubah tanpa mengubah payroll lama.

CREATE TABLE

```
CREATE TABLE pegawai (
  KodePegawai CHAR(6) PRIMARY KEY,
  KodeDepartemen CHAR(5) NOT NULL,
  NamaPegawai VARCHAR(100) NOT NULL,
  Email VARCHAR(120) NOT NULL UNIQUE,
  Jabatan VARCHAR(80) NOT NULL,
  TanggalMasuk DATE NOT NULL,
  StatusPegawai ENUM('AKTIF', 'CUTI', 'NONAKTIF') NOT NULL DEFAULT 'AKTIF',
  GajiPokok DECIMAL(15,2) NOT NULL,
  NamaBank VARCHAR(50) NOT NULL,
  NoRekening VARCHAR(34) NOT NULL,
  CONSTRAINT ck_pegawai_gaji CHECK (GajiPokok >= 0),
  CONSTRAINT fk_pegawai_departemen FOREIGN KEY (KodeDepartemen)
    REFERENCES departemen(KodeDepartemen) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO pegawai
  (KodePegawai, KodeDepartemen, NamaPegawai, Email, Jabatan, TanggalMasuk,
   StatusPegawai, GajiPokok, NamaBank, NoRekening) VALUES
  ('EMP001', 'DPT01', 'Budi Santoso', 'budi@example.test', 'IT Manager',
   '2023-01-10', 'AKTIF', 28500000, 'Bank Contoh A', 'DEMO-REK-001'),
  ('EMP002', 'DPT02', 'Andi Wijaya', 'andi@example.test', 'Operations Manager',
   '2023-03-15', 'AKTIF', 22000000, 'Bank Contoh B', 'DEMO-REK-002'),
  ('EMP003', 'DPT03', 'Siti Rahmawati', 'siti@example.test', 'Finance Manager',
   '2023-05-01', 'AKTIF', 24000000, 'Bank Contoh A', 'DEMO-REK-003'),
  ('EMP004', 'DPT04', 'Rina Kusuma', 'rina@example.test', 'HR Manager',
   '2024-02-05', 'AKTIF', 18000000, 'Bank Contoh C', 'DEMO-REK-004'),
  ('EMP005', 'DPT05', 'Joko Anwar', 'joko@example.test', 'Warehouse Supervisor',
   '2024-06-10', 'AKTIF', 16000000, 'Bank Contoh B', 'DEMO-REK-005');
```

pengguna

Akun dan peran pengguna

PK KodePengguna; FK pegawai unik. Empat peran; lima akun contoh. AuthUID untuk identitas eksternal.

CREATE TABLE

```
CREATE TABLE pengguna (
  KodePengguna CHAR(6) PRIMARY KEY,
  KodePegawai CHAR(6) NOT NULL UNIQUE,
  Username VARCHAR(50) NOT NULL UNIQUE,
  AuthUID VARCHAR(128) NOT NULL UNIQUE,
  Peran ENUM('SUPER_ADMIN', 'WAREHOUSE_MANAGER',
             'FINANCE_MANAGER', 'HR_MANAGER') NOT NULL,
  Aktif BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT ck_pengguna_aktif CHECK (Aktif IN (0, 1)),
  CONSTRAINT fk_pengguna_pegawai FOREIGN KEY (KodePegawai)
    REFERENCES pegawai(KodePegawai) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO pengguna
  (KodePengguna, KodePegawai, Username, AuthUID, Peran, Aktif) VALUES
  ('USR001', 'EMP001', 'admin.demo', 'demo-auth-001', 'SUPER_ADMIN', TRUE),
  ('USR002', 'EMP002', 'gudang.demo', 'demo-auth-002', 'WAREHOUSE_MANAGER', TRUE),
  ('USR003', 'EMP003', 'finance.demo', 'demo-auth-003', 'FINANCE_MANAGER', TRUE),
  ('USR004', 'EMP004', 'hr.demo', 'demo-auth-004', 'HR_MANAGER', TRUE),
  ('USR005', 'EMP005', 'gudang2.demo', 'demo-auth-005', 'WAREHOUSE_MANAGER', TRUE);
```

supplier

Pemasok barang

PK KodeSupplier; Email unik. Kontak contoh, bukan data pemasok nyata.

CREATE TABLE

```
CREATE TABLE supplier (
  KodeSupplier CHAR(6) PRIMARY KEY,
  NamaSupplier VARCHAR(120) NOT NULL,
  Email VARCHAR(120) NOT NULL UNIQUE,
  Telepon VARCHAR(20) NOT NULL,
  Alamat VARCHAR(255) NOT NULL
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO supplier (KodeSupplier, NamaSupplier, Email, Telepon, Alamat) VALUES
  ('SUP001', 'PT Contoh Komputasi', 'komputasi@example.test', 'DEMO-TEL-001',
   'Alamat contoh supplier 1, Jakarta'),
  ('SUP002', 'PT Contoh Server', 'server@example.test', 'DEMO-TEL-002',
   'Alamat contoh supplier 2, Bandung'),
  ('SUP003', 'PT Contoh Storage', 'storage@example.test', 'DEMO-TEL-003',
   'Alamat contoh supplier 3, Surabaya'),
  ('SUP004', 'PT Contoh Jaringan', 'jaringan@example.test', 'DEMO-TEL-004',
   'Alamat contoh supplier 4, Semarang'),
  ('SUP005', 'PT Contoh Daya', 'daya@example.test', 'DEMO-TEL-005',
   'Alamat contoh supplier 5, Yogyakarta');
```

kategori_produk

Pengelompokan produk

PK KodeKategori; NamaKategori unik. Menjadi referensi master produk.

CREATE TABLE

```
CREATE TABLE kategori_produk (
  KodeKategori CHAR(6) PRIMARY KEY,
  NamaKategori VARCHAR(60) NOT NULL UNIQUE,
  Keterangan VARCHAR(160) NOT NULL
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO kategori_produk (KodeKategori, NamaKategori, Keterangan) VALUES
  ('KAT001', 'GPU', 'Perangkat komputasi grafis dan AI'),
  ('KAT002', 'Server', 'Perangkat server perusahaan'),
  ('KAT003', 'Storage', 'Media penyimpanan data'),
  ('KAT004', 'Networking', 'Perangkat jaringan'),
  ('KAT005', 'Power', 'Perangkat pendukung daya');
```

produk

Master inventori

PK KodeProduk; FK kategori; SKU unik. StokMin dan Stok berupa bilangan nonnegatif.

CREATE TABLE

```
CREATE TABLE produk (
  KodeProduk CHAR(6) PRIMARY KEY,
  KodeKategori CHAR(6) NOT NULL,
  SKU VARCHAR(40) NOT NULL UNIQUE,
  NamaProduk VARCHAR(120) NOT NULL,
  Satuan VARCHAR(20) NOT NULL,
  HargaSatuan DECIMAL(15,2) NOT NULL,
  StokMin INT UNSIGNED NOT NULL DEFAULT 0,
  Stok INT UNSIGNED NOT NULL DEFAULT 0,
  CONSTRAINT ck_produk_harga CHECK (HargaSatuan >= 0),
  CONSTRAINT fk_produk_kategori FOREIGN KEY (KodeKategori)
    REFERENCES kategori_produk(KodeKategori) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO produk
  (KodeProduk, KodeKategori, SKU, NamaProduk, Satuan, HargaSatuan, StokMin, Stok)
VALUES
  ('PRD001', 'KAT001', 'NV-H100-TC', 'GPU H100', 'Unit', 250000000, 10, 142),
  ('PRD002', 'KAT002', 'RS-W1-PRO', 'Rack Server', 'Unit', 50000000, 10, 58),
  ('PRD003', 'KAT003', 'STR-NVME-8TB', 'NVMe 8TB', 'Unit', 12000000, 15, 8),
  ('PRD004', 'KAT004', 'SW-10GBE-L3', 'Switch 10GbE', 'Unit', 8500000, 15, 86),
  ('PRD005', 'KAT005', 'UPS-3KVA-RT', 'UPS 3kVA', 'Unit', 6500000, 10, 45);
```

purchase_order

Header pengadaan

PK KodePO; FK supplier, departemen, pembuat. Total = Subtotal - Diskon + Pajak - Potongan.

CREATE TABLE

```
CREATE TABLE purchase_order (
  KodePO CHAR(7) PRIMARY KEY,
  KodeSupplier CHAR(6) NOT NULL,
  KodeDepartemen CHAR(5) NOT NULL,
  DibuatOleh CHAR(6) NOT NULL,
  TanggalPO DATE NOT NULL,
  TanggalKirim DATE NOT NULL,
  StatusPO ENUM('MENUNGGU', 'DISETUJUI', 'DITOLAK',
                'DITERIMA', 'SELESAI') NOT NULL DEFAULT 'MENUNGGU',
  Subtotal DECIMAL(15,2) NOT NULL,
  Diskon DECIMAL(15,2) NOT NULL DEFAULT 0,
  Pajak DECIMAL(15,2) NOT NULL DEFAULT 0,
  Potongan DECIMAL(15,2) NOT NULL DEFAULT 0,
  Total DECIMAL(15,2) GENERATED ALWAYS AS
    (Subtotal - Diskon + Pajak - Potongan) STORED,
  CONSTRAINT ck_po_tanggal CHECK (TanggalKirim >= TanggalPO),
  CONSTRAINT ck_po_nominal CHECK
    (Subtotal >= 0 AND Diskon >= 0 AND Diskon <= Subtotal AND Pajak >= 0
     AND Potongan >= 0 AND Potongan <= Subtotal - Diskon + Pajak),
  CONSTRAINT fk_po_supplier FOREIGN KEY (KodeSupplier)
    REFERENCES supplier(KodeSupplier) ON UPDATE RESTRICT ON DELETE RESTRICT,
  CONSTRAINT fk_po_departemen FOREIGN KEY (KodeDepartemen)
    REFERENCES departemen(KodeDepartemen) ON UPDATE RESTRICT ON DELETE RESTRICT,
  CONSTRAINT fk_po_pembuat FOREIGN KEY (DibuatOleh)
    REFERENCES pengguna(KodePengguna) ON UPDATE RESTRICT ON DELETE RESTRICT,
  INDEX idx_po_tanggal_status (TanggalPO, StatusPO)
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO purchase_order
  (KodePO, KodeSupplier, KodeDepartemen, DibuatOleh, TanggalPO, TanggalKirim,
   StatusPO, Subtotal, Diskon, Pajak, Potongan) VALUES
  ('PO00001', 'SUP001', 'DPT01', 'USR002', '2026-10-01', '2026-10-06',
   'SELESAI', 500000000, 0, 55000000, 0),
  ('PO00002', 'SUP002', 'DPT02', 'USR002', '2026-10-02', '2026-10-07',
   'SELESAI', 150000000, 5000000, 15950000, 1500000),
  ('PO00003', 'SUP003', 'DPT03', 'USR002', '2026-10-03', '2026-10-08',
   'SELESAI', 60000000, 0, 6600000, 0),
  ('PO00004', 'SUP004', 'DPT04', 'USR005', '2026-10-04', '2026-10-09',
   'SELESAI', 34000000, 1000000, 3630000, 0),
  ('PO00005', 'SUP005', 'DPT05', 'USR005', '2026-10-05', '2026-10-10',
   'SELESAI', 39000000, 0, 4290000, 0);
```

detail_purchase_order

Rincian item pengadaan

PK gabungan KodePO + KodeProduk. HargaSatuan menyimpan harga historis; Subtotal = Quantity x HargaSatuan.

CREATE TABLE

```
CREATE TABLE detail_purchase_order (
  KodePO CHAR(7) NOT NULL,
  KodeProduk CHAR(6) NOT NULL,
  Quantity INT UNSIGNED NOT NULL,
  HargaSatuan DECIMAL(15,2) NOT NULL,
  Subtotal DECIMAL(15,2) GENERATED ALWAYS AS (Quantity * HargaSatuan) STORED,
  PRIMARY KEY (KodePO, KodeProduk),
  CONSTRAINT ck_detail_po_qty CHECK (Quantity > 0),
  CONSTRAINT ck_detail_po_harga CHECK (HargaSatuan >= 0),
  CONSTRAINT fk_detail_po_header FOREIGN KEY (KodePO)
    REFERENCES purchase_order(KodePO) ON UPDATE RESTRICT ON DELETE RESTRICT,
  CONSTRAINT fk_detail_po_produk FOREIGN KEY (KodeProduk)
    REFERENCES produk(KodeProduk) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO detail_purchase_order
  (KodePO, KodeProduk, Quantity, HargaSatuan) VALUES
  ('PO00001', 'PRD001', 2, 250000000),
  ('PO00002', 'PRD002', 3, 50000000),
  ('PO00003', 'PRD003', 5, 12000000),
  ('PO00004', 'PRD004', 4, 8500000),
  ('PO00005', 'PRD005', 6, 6500000);
```

persetujuan_po

Keputusan pengadaan

PK KodePersetujuan; KodePO unik. Maksimal satu keputusan persetujuan/penolakan per PO.

CREATE TABLE

```
CREATE TABLE persetujuan_po (
  KodePersetujuan CHAR(7) PRIMARY KEY,
  KodePO CHAR(7) NOT NULL UNIQUE,
  DisetujuiOleh CHAR(6) NOT NULL,
  TanggalPersetujuan DATETIME NOT NULL,
  Keputusan ENUM('DISETUJUI', 'DITOLAK') NOT NULL,
  Catatan VARCHAR(160) NOT NULL,
  CONSTRAINT fk_persetujuan_po FOREIGN KEY (KodePO)
    REFERENCES purchase_order(KodePO) ON UPDATE RESTRICT ON DELETE RESTRICT,
  CONSTRAINT fk_persetujuan_pengguna FOREIGN KEY (DisetujuiOleh)
    REFERENCES pengguna(KodePengguna) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO persetujuan_po
  (KodePersetujuan, KodePO, DisetujuiOleh, TanggalPersetujuan, Keputusan, Catatan)
VALUES
  ('APV0001', 'PO00001', 'USR001', '2026-10-02 09:00:00',
   'DISETUJUI', 'Sesuai kebutuhan Engineering'),
  ('APV0002', 'PO00002', 'USR003', '2026-10-03 09:15:00',
   'DISETUJUI', 'Sesuai anggaran Operasional'),
  ('APV0003', 'PO00003', 'USR003', '2026-10-04 09:30:00',
   'DISETUJUI', 'Dokumen pengadaan lengkap'),
  ('APV0004', 'PO00004', 'USR001', '2026-10-05 09:45:00',
   'DISETUJUI', 'Perangkat jaringan disetujui'),
  ('APV0005', 'PO00005', 'USR003', '2026-10-06 10:00:00',
   'DISETUJUI', 'Pengadaan daya gudang disetujui');
```

penerimaan_barang

Dokumen penerimaan

PK sekaligus FK KodePO; KodeTerima unik. Maksimal satu dokumen penerimaan untuk setiap PO.

CREATE TABLE

```
CREATE TABLE penerimaan_barang (
  KodePO CHAR(7) PRIMARY KEY,
  KodeTerima CHAR(7) NOT NULL UNIQUE,
  DiterimaOleh CHAR(6) NOT NULL,
  TanggalTerima DATE NOT NULL,
  NoSuratJalan VARCHAR(40) NOT NULL UNIQUE,
  StatusTerima ENUM('SESUAI', 'SELISIH') NOT NULL,
  CONSTRAINT fk_penerimaan_po FOREIGN KEY (KodePO)
    REFERENCES purchase_order(KodePO) ON UPDATE RESTRICT ON DELETE RESTRICT,
  CONSTRAINT fk_penerimaan_pengguna FOREIGN KEY (DiterimaOleh)
    REFERENCES pengguna(KodePengguna) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO penerimaan_barang
  (KodePO, KodeTerima, DiterimaOleh, TanggalTerima, NoSuratJalan, StatusTerima)
VALUES
  ('PO00001', 'GRN0001', 'USR002', '2026-10-06', 'SJ-DEMO-001', 'SESUAI'),
  ('PO00002', 'GRN0002', 'USR002', '2026-10-07', 'SJ-DEMO-002', 'SESUAI'),
  ('PO00003', 'GRN0003', 'USR005', '2026-10-08', 'SJ-DEMO-003', 'SESUAI'),
  ('PO00004', 'GRN0004', 'USR005', '2026-10-09', 'SJ-DEMO-004', 'SESUAI'),
  ('PO00005', 'GRN0005', 'USR005', '2026-10-10', 'SJ-DEMO-005', 'SESUAI');
```

detail_penerimaan

Rincian barang diterima

PK gabungan KodePO + KodeProduk. FK gabungan mengacu item dari PO yang sama.

CREATE TABLE

```
CREATE TABLE detail_penerimaan (
  KodePO CHAR(7) NOT NULL,
  KodeProduk CHAR(6) NOT NULL,
  QtyDiterima INT UNSIGNED NOT NULL,
  Kondisi ENUM('BAIK', 'RUSAK') NOT NULL,
  Catatan VARCHAR(160) NOT NULL,
  PRIMARY KEY (KodePO, KodeProduk),
  CONSTRAINT ck_detail_terima_qty CHECK (QtyDiterima > 0),
  CONSTRAINT fk_detail_terima_header FOREIGN KEY (KodePO)
    REFERENCES penerimaan_barang(KodePO) ON UPDATE RESTRICT ON DELETE RESTRICT,
  CONSTRAINT fk_detail_terima_item FOREIGN KEY (KodePO, KodeProduk)
    REFERENCES detail_purchase_order(KodePO, KodeProduk)
    ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO detail_penerimaan
  (KodePO, KodeProduk, QtyDiterima, Kondisi, Catatan) VALUES
  ('PO00001', 'PRD001', 2, 'BAIK', 'Jumlah sesuai PO'),
  ('PO00002', 'PRD002', 3, 'BAIK', 'Jumlah sesuai PO'),
  ('PO00003', 'PRD003', 5, 'BAIK', 'Jumlah sesuai PO'),
  ('PO00004', 'PRD004', 4, 'BAIK', 'Jumlah sesuai PO'),
  ('PO00005', 'PRD005', 6, 'BAIK', 'Jumlah sesuai PO');
```

pembayaran_po

Pembayaran pengadaan

PK KodeBayar; KodePO unik dan mengacu penerimaan. Semua transaksi contoh bersifat simulasi.

CREATE TABLE

```
CREATE TABLE pembayaran_po (
  KodeBayar CHAR(7) PRIMARY KEY,
  KodePO CHAR(7) NOT NULL UNIQUE,
  DibayarOleh CHAR(6) NOT NULL,
  TanggalBayar DATETIME NOT NULL,
  MetodeBayar ENUM('TRANSFER_SIMULASI', 'VA_SIMULASI', 'QRIS_SIMULASI') NOT NULL,
  Nominal DECIMAL(15,2) NOT NULL,
  Referensi VARCHAR(60) NOT NULL UNIQUE,
  StatusBayar ENUM('SIMULASI_BERHASIL', 'SIMULASI_GAGAL') NOT NULL,
  CONSTRAINT ck_pembayaran_nominal CHECK (Nominal > 0),
  CONSTRAINT fk_pembayaran_penerimaan FOREIGN KEY (KodePO)
    REFERENCES penerimaan_barang(KodePO) ON UPDATE RESTRICT ON DELETE RESTRICT,
  CONSTRAINT fk_pembayaran_pengguna FOREIGN KEY (DibayarOleh)
    REFERENCES pengguna(KodePengguna) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO pembayaran_po
  (KodeBayar, KodePO, DibayarOleh, TanggalBayar, MetodeBayar, Nominal,
   Referensi, StatusBayar) VALUES
  ('PAY0001', 'PO00001', 'USR003', '2026-10-07 13:00:00', 'TRANSFER_SIMULASI',
   555000000, 'SIM-BAYAR-001', 'SIMULASI_BERHASIL'),
  ('PAY0002', 'PO00002', 'USR003', '2026-10-08 13:00:00', 'VA_SIMULASI',
   159450000, 'SIM-BAYAR-002', 'SIMULASI_BERHASIL'),
  ('PAY0003', 'PO00003', 'USR003', '2026-10-09 13:00:00', 'TRANSFER_SIMULASI',
   66600000, 'SIM-BAYAR-003', 'SIMULASI_BERHASIL'),
  ('PAY0004', 'PO00004', 'USR001', '2026-10-10 13:00:00', 'QRIS_SIMULASI',
   36630000, 'SIM-BAYAR-004', 'SIMULASI_BERHASIL'),
  ('PAY0005', 'PO00005', 'USR003', '2026-10-11 13:00:00', 'VA_SIMULASI',
   43290000, 'SIM-BAYAR-005', 'SIMULASI_BERHASIL');
```

payroll

Riwayat gaji per periode

PK KodePayroll; unik per pegawai-periode. Komponen historis dan GajiBersih otomatis; periode memakai tanggal satu.

CREATE TABLE

```
CREATE TABLE payroll (
  KodePayroll CHAR(7) PRIMARY KEY,
  KodePegawai CHAR(6) NOT NULL,
  DicatatOleh CHAR(6) NOT NULL,
  Periode DATE NOT NULL,
  TanggalCatat DATETIME NOT NULL,
  GajiPokok DECIMAL(15,2) NOT NULL,
  TunjanganJabatan DECIMAL(15,2) NOT NULL DEFAULT 0,
  TunjanganTransport DECIMAL(15,2) NOT NULL DEFAULT 0,
  Bonus DECIMAL(15,2) NOT NULL DEFAULT 0,
  BPJSKesehatan DECIMAL(15,2) NOT NULL DEFAULT 0,
  BPJSKetenagakerjaan DECIMAL(15,2) NOT NULL DEFAULT 0,
  PPh21 DECIMAL(15,2) NOT NULL DEFAULT 0,
  GajiBersih DECIMAL(15,2) GENERATED ALWAYS AS
    (GajiPokok + TunjanganJabatan + TunjanganTransport + Bonus
     - BPJSKesehatan - BPJSKetenagakerjaan - PPh21) STORED,
  UNIQUE KEY uq_payroll_pegawai_periode (KodePegawai, Periode),
  CONSTRAINT ck_payroll_periode CHECK (DAYOFMONTH(Periode) = 1),
  CONSTRAINT ck_payroll_nominal CHECK
    (GajiPokok >= 0 AND TunjanganJabatan >= 0 AND TunjanganTransport >= 0
     AND Bonus >= 0 AND BPJSKesehatan >= 0 AND BPJSKetenagakerjaan >= 0
     AND PPh21 >= 0 AND GajiBersih >= 0),
  CONSTRAINT fk_payroll_pegawai FOREIGN KEY (KodePegawai)
    REFERENCES pegawai(KodePegawai) ON UPDATE RESTRICT ON DELETE RESTRICT,
  CONSTRAINT fk_payroll_pencatat FOREIGN KEY (DicatatOleh)
    REFERENCES pengguna(KodePengguna) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO payroll
  (KodePayroll, KodePegawai, DicatatOleh, Periode, TanggalCatat, GajiPokok,
   TunjanganJabatan, TunjanganTransport, Bonus, BPJSKesehatan,
   BPJSKetenagakerjaan, PPh21) VALUES
  ('PAYR001', 'EMP001', 'USR004', '2026-10-01', '2026-10-20 10:00:00',
   28500000, 4500000, 1500000, 2500000, 285000, 855000, 1850000),
  ('PAYR002', 'EMP002', 'USR004', '2026-10-01', '2026-10-20 10:10:00',
   22000000, 3200000, 1400000, 1200000, 220000, 660000, 1250000),
  ('PAYR003', 'EMP003', 'USR004', '2026-10-01', '2026-10-20 10:20:00',
   24000000, 3800000, 1200000, 1500000, 240000, 720000, 1420000),
  ('PAYR004', 'EMP004', 'USR001', '2026-10-01', '2026-10-20 10:30:00',
   18000000, 2000000, 1000000, 500000, 180000, 540000, 800000),
  ('PAYR005', 'EMP005', 'USR004', '2026-10-01', '2026-10-20 10:40:00',
   16000000, 1500000, 1000000, 750000, 160000, 480000, 700000);
```

anggaran_departemen

Anggaran per kuartal

PK KodeAnggaran; unik per departemen-tahun-kuartal. Sisa anggaran dihitung melalui query, bukan disimpan ulang.

CREATE TABLE

```
CREATE TABLE anggaran_departemen (
  KodeAnggaran CHAR(7) PRIMARY KEY,
  KodeDepartemen CHAR(5) NOT NULL,
  Tahun SMALLINT UNSIGNED NOT NULL,
  Kuartal TINYINT UNSIGNED NOT NULL,
  NominalAnggaran DECIMAL(15,2) NOT NULL,
  UNIQUE KEY uq_anggaran_periode (KodeDepartemen, Tahun, Kuartal),
  CONSTRAINT ck_anggaran_tahun CHECK (Tahun BETWEEN 2000 AND 2100),
  CONSTRAINT ck_anggaran_kuartal CHECK (Kuartal BETWEEN 1 AND 4),
  CONSTRAINT ck_anggaran_nominal CHECK (NominalAnggaran > 0),
  CONSTRAINT fk_anggaran_departemen FOREIGN KEY (KodeDepartemen)
    REFERENCES departemen(KodeDepartemen) ON UPDATE RESTRICT ON DELETE RESTRICT
) ENGINE=InnoDB;
```

INSERT - 5 DATA CONTOH

```
INSERT INTO anggaran_departemen
  (KodeAnggaran, KodeDepartemen, Tahun, Kuartal, NominalAnggaran) VALUES
  ('BGT0001', 'DPT01', 2026, 4, 1000000000),
  ('BGT0002', 'DPT02', 2026, 4, 300000000),
  ('BGT0003', 'DPT03', 2026, 4, 150000000),
  ('BGT0004', 'DPT04', 2026, 4, 100000000),
  ('BGT0005', 'DPT05', 2026, 4, 120000000);
```

Verifikasi data (1/3)

Jumlah data, relasi transaksi, dan keseimbangan subtotal. Jalankan setelah USE inventora_tubes;

1. Harus 14 tabel, masing-masing JumlahData = 5 (total 70 baris).

```
SELECT 'departemen' AS NamaTabel, COUNT(*) AS JumlahData FROM departemen
UNION ALL SELECT 'pegawai', COUNT(*) FROM pegawai
UNION ALL SELECT 'pengguna', COUNT(*) FROM pengguna
UNION ALL SELECT 'supplier', COUNT(*) FROM supplier
UNION ALL SELECT 'kategori_produk', COUNT(*) FROM kategori_produk
UNION ALL SELECT 'produk', COUNT(*) FROM produk
UNION ALL SELECT 'purchase_order', COUNT(*) FROM purchase_order
UNION ALL SELECT 'detail_purchase_order', COUNT(*) FROM detail_purchase_order
UNION ALL SELECT 'persetujuan_po', COUNT(*) FROM persetujuan_po
UNION ALL SELECT 'penerimaan_barang', COUNT(*) FROM penerimaan_barang
UNION ALL SELECT 'detail_penerimaan', COUNT(*) FROM detail_penerimaan
UNION ALL SELECT 'pembayaran_po', COUNT(*) FROM pembayaran_po
UNION ALL SELECT 'payroll', COUNT(*) FROM payroll
UNION ALL SELECT 'anggaran_departemen', COUNT(*) FROM anggaran_departemen;
```

2. Relasi header-detail dan supplier, total otomatis, nominal pembayaran.

```
SELECT po.KodePO, s.NamaSupplier, COUNT(d.KodeProduk) AS JumlahJenis,
       SUM(d.Quantity) AS JumlahUnit, po.Total, b.Nominal AS Dibayar
FROM purchase_order po
JOIN supplier s ON s.KodeSupplier = po.KodeSupplier
JOIN detail_purchase_order d ON d.KodePO = po.KodePO
JOIN pembayaran_po b ON b.KodePO = po.KodePO
GROUP BY po.KodePO, s.NamaSupplier, po.Total, b.Nominal
ORDER BY po.KodePO;
```

3. Harus kosong: subtotal header berbeda dari jumlah detail.

```
SELECT po.KodePO
FROM purchase_order po
LEFT JOIN detail_purchase_order d ON d.KodePO = po.KodePO
GROUP BY po.KodePO, po.Subtotal
HAVING po.Subtotal <> COALESCE(SUM(d.Subtotal), 0);
```

Hasil: 14 tabel dengan masing-masing lima baris; lima PO memiliki pembayaran yang sama dengan totalnya; query ketidaksesuaian subtotal tidak mengembalikan baris.

Verifikasi data (2/3)

Konsistensi pembayaran, penerimaan, dan riwayat payroll. Jalankan setelah USE inventora_tubes;

4. Harus kosong: pembayaran berbeda dari total atau penerimaan belum sesuai.

```
SELECT b.KodeBayar
FROM pembayaran_po b
JOIN purchase_order po ON po.KodePO = b.KodePO
JOIN penerimaan_barang r ON r.KodePO = b.KodePO
WHERE b.StatusBayar = 'SIMULASI_BERHASIL'
  AND (b.Nominal <> po.Total OR r.StatusTerima <> 'SESUAI');
```

5. Harus kosong: catatan penerimaan berstatus SESUAI tetapi isi tidak sesuai.

```
SELECT d.KodePO, d.KodeProduk
FROM detail_purchase_order d
JOIN penerimaan_barang h ON h.KodePO = d.KodePO
LEFT JOIN detail_penerimaan r
  ON r.KodePO = d.KodePO AND r.KodeProduk = d.KodeProduk
WHERE h.StatusTerima = 'SESUAI'
  AND (r.KodeProduk IS NULL OR r.QtyDiterima <> d.Quantity OR r.Kondisi <> 'BAIK');
```

6. Payroll per pegawai, periode dan nilai bersih yang dihasilkan otomatis.

```
SELECT p.KodePayroll, e.NamaPegawai, DATE_FORMAT(p.Periode, '%Y-%m') AS Periode,
       p.GajiBersih
FROM payroll p
JOIN pegawai e ON e.KodePegawai = p.KodePegawai
ORDER BY p.KodePayroll;
```

Hasil: query ketidaksesuaian pembayaran dan penerimaan kosong. Payroll menghasilkan lima baris Oktober 2026 dengan total GajiBersih Rp125.690.000,00.

Verifikasi data (3/3)

Peringatan stok minimum dan anggaran terhadap komitmen PO. Jalankan setelah USE inventora_tubes;

7. Produk di bawah batas minimum: PRD003 / NVMe 8TB.

```
SELECT KodeProduk, NamaProduk, Stok, StokMin
FROM produk WHERE Stok < StokMin;
```

8. Anggaran dibandingkan komitmen pengadaan pada kuartal yang sama.

```
SELECT a.KodeAnggaran, d.NamaDepartemen, a.Tahun, a.Kuartal,
       a.NominalAnggaran, COALESCE(SUM(po.Total), 0) AS NilaiPO,
       a.NominalAnggaran - COALESCE(SUM(po.Total), 0) AS SisaAnggaran
FROM anggaran_departemen a
JOIN departemen d ON d.KodeDepartemen = a.KodeDepartemen
LEFT JOIN purchase_order po
  ON po.KodeDepartemen = a.KodeDepartemen
  AND YEAR(po.TanggalPO) = a.Tahun AND QUARTER(po.TanggalPO) = a.Kuartal
  AND po.StatusPO <> 'DITOLAK'
GROUP BY a.KodeAnggaran, d.NamaDepartemen, a.Tahun, a.Kuartal, a.NominalAnggaran
ORDER BY a.KodeAnggaran;
```

DEPARTEMEN	ANGGARAN Q4 2026	NILAI PO	SISA ANGGARAN
Engineering dan IT	1.000.000.000	555.000.000	445.000.000
Operasional	300.000.000	159.450.000	140.550.000
Keuangan	150.000.000	66.600.000	83.400.000
Sumber Daya Manusia	100.000.000	36.630.000	63.370.000
Pergudangan	120.000.000	43.290.000	76.710.000

Semua angka tabel dalam rupiah. NilaiPO adalah komitmen PO selain yang ditolak pada kuartal yang sama, bukan hanya arus kas yang sudah dibayar. Produk di bawah minimum: PRD003 / NVMe 8TB, stok 8 dari batas minimum 15.

20 pemeriksaan lulus

Dijalankan pada MySQL Community Server 8.0.41, 21 Oktober 2026, melalui instance sementara terpisah dari data aplikasi.

NO.	PEMERIKSAAN	HASIL
01	CREATE dan INSERT lengkap	LULUS
02	Jumlah tabel dan data	LULUS
03	Relasi foreign key	LULUS
04	Agregat nilai contoh	LULUS
05	Subtotal header sama dengan detail	LULUS
06	Pembayaran penuh dan penerimaan	LULUS
07	Isi penerimaan sama dengan pesanan	LULUS
08	Urutan tanggal transaksi contoh	LULUS
09	Peran pelaku pada data contoh	LULUS
10	Deteksi stok minimum	LULUS
11	Produk dengan kategori tidak ada	LULUS
12	Harga produk negatif	LULUS
13	Quantity pesanan nol	LULUS
14	Payroll pegawai-periode ganda	LULUS
15	Periode payroll bukan tanggal satu	LULUS
16	Penerimaan produk di luar PO	LULUS
17	Pembayaran kedua untuk PO sama	LULUS
18	Tanggal kirim sebelum PO	LULUS
19	Data tetap utuh setelah uji negatif	LULUS
20	Semua query verifikasi	LULUS

Delapan uji negatif: pelanggaran FK ditolak dengan ERROR 1452, pelanggaran CHECK dengan ERROR 3819, dan duplikasi key dengan ERROR 1062. Percobaan dijalankan dalam transaksi yang dibatalkan; semua tabel tetap berisi lima baris.

Agregat contoh: total PO dan pembayaran Rp860.970.000,00; payroll Oktober Rp125.690.000,00; jumlah stok 339 unit. Query subtotal, pembayaran, dan penerimaan tidak menemukan ketidaksesuaian.

Pemeriksaan peran dan urutan tanggal membuktikan konsistensi data contoh. FK sendiri tidak menegakkan hak akses ataupun seluruh aturan proses bisnis. Sumber SQL, query, dan rincian hasil uji disertakan dalam paket database/tubes.